

Fiche de démonstration — Oral E6 · ColorRoom

· Téo Trompier (E2)

Après les 20 min de présentation → 20 min de démonstration. Cette fiche couvre, point par point, **exactement** ce qu'a demandé M. Delbosc. Coche mentalement chaque exigence pendant que tu démontres.

#	Exigence du jury	Où c'est traité ici
1	Présenter le système réel = diagramme de déploiement	§1
2	Test(s) de recette : CU + acteurs, conditions initiales, scénario, validation CDC	§2 (3 fiches de recette)
3	Outils de gestion du projet et du code + mise en œuvre	§3
4	Situer ma contribution dans le projet global	§4
5	Constituants de ma partie + interactions avec les camarades (tests d'intégration)	§5
6	Montrer/expliquer mon code et son évolution	§6

 **Plan minuté (20 min) : vidéo 2 min** (système réel) · §1 réel/déploiement 1 min · §2 recette 7 min · §3 outils 3 min · §4 contribution 2 min · §5 intégration 3 min · §6 code & évolution 2 min.

 **Avoir une vidéo de secours** de chaque test de recette, au cas où le Pi / le CS-160 / le Wi-Fi flanche.

 **À taper en tout premier sur la tablette devant le jury** : <http://172.17.40.39/> (Wi-Fi local **ColorRoom_WiFi**, fonctionne depuis n'importe quel appareil de la salle).

0. La vidéo de 2 min — slide-pont (fin présentation → début démo)

La vidéo est la **dernière slide du diaporama** (slide 39, embarquée dans le `.pptx`). Elle **clôt la présentation** et **ouvre la démonstration** : je la lance à la toute fin du parlé, puis j'enchaîne sur les manip live.

1. La vraie **ColorRoom** (2 cellules, 42 plaques) est à **LUMEN**, pas dans la salle : la vidéo est **le seul moyen de montrer le système réel complet** exigé par le jury (« éléments réels correspondant au diagramme de déploiement »).
2. Elle **plante le décor** (matériel, dalles qui s'allument) avant les manip live.
3. **Filet de sécurité** : si le Pi/CS-160/Wi-Fi bug en live, le jury a déjà vu le système marcher.

À dire en lançant la vidéo (15 s) : « Voici en 2 minutes le **système réel** installé à LUMEN : les deux cellules, les plaques pilotées en **RS-485**, et l'application en service. » → puis enchaîner sur les **tests de recette live**.

 **Lecture** : clic sur la vidéo dans PowerPoint (ou auto en diaporama). Garder [video_demo.mp4](#) à côté en secours.

1. Le système réel = le diagramme de déploiement (3 min)

À faire : montrer le matériel posé sur la table et le pointer du doigt en suivant le diagramme de déploiement.

Diagramme de déploiement (rappel) → correspondance physique :

Nœud du diagramme	Élément réel à montrer
Raspberry Pi 5	la carte (le « serveur » de la salle), alimentée, en réseau
Docker (sur le Pi)	<code>docker compose ps</code> → 4 conteneurs : <code>color-room</code> , <code>portainer</code> , <code>color-room-ollama</code> , <code>ollama-pull</code>
App Next.js	navigateur → <code>http://172.17.40.39/</code> (ou <code>http://<ip-du-pi>:8080</code>)
supervision.exe	le pont matériel : reçoit mon API et émet des trames RS-485 vers les plaques
Dalles LED	les 42 plaques (21/cellule, 2 cellules) ; une plaque = 2360 LED , 32 canaux (24 spectres étroits + 8 blancs), pilotée en RS-485
CS-160	le colorimètre Konica Minolta branché en USB sur le Pi
Clients Wi-Fi	tablette + téléphone connectés au Wi-Fi local ColorRoom_WiFi
Ollama	conteneur d'IA locale (assistant de l'éditeur, 100 % hors-ligne)

Phrase d'accroche : « Voici le diagramme de déploiement, et voici les éléments réels qui lui correspondent : un Raspberry Pi 5 qui héberge, dans Docker, l'application Next.js, le pont matériel supervision et l'IA locale ; les **42 dalles** sont pilotées en **bus RS-485**, le colorimètre **CS-160 en USB** ; les utilisateurs se connectent en **Wi-Fi local (ColorRoom_WiFi)** sur `http://172.17.40.39/`. **Tout est hors-ligne.** »

Commandes à taper en live (preuve que c'est réel) :

```
docker compose ps          # les conteneurs tournent
curl -s http://172.17.40.39/api/health  # l'app répond {"ok":true}
ip a | grep wlan           # le Pi diffuse bien le réseau de la salle
```

2. Tests issus de la recette (7 min) — le cœur de la démo

Pour **chaque** test, je verbalise les 5 temps imposés : **(a) cas d'utilisation + acteurs** → **(b) conditions initiales** → **(c) scénario** → **(d) résultat** → **(e) objectif du CDC validé**.

Les cas d'utilisation viennent du **diagramme de cas d'utilisation** : acteurs **Enseignant** et **Apprenant** ; CU ***Jouer***, ***Mesurer (CS-160)***, ***Générer par IA***, ***Créer un jeu***, ***Allumer les dalles***.

Fiche de recette R1 — « Un apprenant joue et les dalles s'allument »

Champ	Contenu
Cas d'utilisation	*Jouer* (qui *inclut* *Allumer les dalles*)
Acteur(s)	Apprenant (primaire) · le matériel (dalles LED) acteur secondaire
Préconditions	Pi démarré, conteneur color-room UP, dalles alimentées, apprenant inscrit/connecté
Scénario nominal	1) Se connecter comme apprenant → 2) ouvrir le catalogue / → 3) lancer Color Speed → 4) une dalle cible s'allume (3D et dalle réelle) → 5) toucher la bonne couleur → 6) le score s'incrémente
Résultat attendu	La couleur affichée à l'écran = la couleur émise par la dalle ; le score est correct ; pas de latence visible
Objectif CDC validé	« Rendre la ColorRoom accessible en initiation par des jeux » + pilotage temps réel des dalles

Pourquoi c'est probant techniquement : le navigateur ne parle **jamais** directement au matériel. Le clic → route API **/api/supervision/batch** → **sémaphore** **HW_CONCURRENCY=2** + file d'attente (supervision est quasi-série) → envoi des **32 canaux** avec **AbortController** (timeout). Couleur écran = couleur dalle via **CHANNEL_PROFILES** (chaque canal ↔ sa longueur d'onde).

Fiche de recette R2 — « Mesure colorimétrique au CS-160 »

Champ	Contenu
Cas d'utilisation	*Mesurer (CS-160)* (qui *inclut* *Allumer les dalles*)
Acteur(s)	Apprenant/Enseignant · colorimètre CS-160 (acteur secondaire)
Préconditions	CS-160 branché en USB, pont /api/cs160 joignable, une dalle cible allumée
Scénario nominal	1) Page Mesure → 2) « Connecter » le CS-160 → 3) allumer une dalle cible → 4) « Mesurer » → 5) lire X Y Z, Lv (cd/m²), x,y → 6) le point s'affiche sur le diagramme CIE 1931
Résultat attendu	Valeurs tristimulus cohérentes ; point CIE positionné ; ΔE calculé vs cible
Objectif CDC validé	Donner accès à la mesure scientifique réelle (colorimétrie) de façon pédagogique

Fiche de recette R3 — « Affrontement contre l'IA / Multijoueur »

Champ	Contenu
Cas d'utilisation	*Jouer* (mode IA et mode multijoueur)
Acteur(s)	Apprenant (×2 en multi : hôte + invité)
Préconditions	App UP ; pour le multi : 2 terminaux sur le Wi-Fi local
Scénario A (IA)	Lancer Puissance 4 , niveau *Légendaire* → poser une menace à 3 → l'IA la bloque (anti-piège)
Scénario B (multi)	Hôte crée une partie → code affiché → invité saisit le code → les 2 jouent, scores synchronisés
Résultat attendu	A : l'IA défend correctement, réponse quasi instantanée · B : état partagé cohérent entre les 2 écrans
Objectif CDC validé	Jeux solo + multijoueur engageants ; robustesse en réseau local

Technique à dire : IA = **minimax + élagage alpha-bêta**, profondeurs **1/2/5/9/12** ; heuristique `scoreWindow` (défense **-170** > **attaque +130**, victoire = 1 000 000, bonus centralité). Multi **sans WebSocket** : état **persisté** dans `crg_mp_sessions.state_json`, lu en **polling** → simple et robuste à déployer sur Pi.

3. Outils de gestion du projet et du code (3 min)

À montrer concrètement (pas juste citer) :

- **Git + GitHub** : `git log --oneline` → **~280 commits**, messages clairs et préfixés (`feat` , `fix` , `perf` , `docs` , `chore`). Montrer la liste des **branches** (`ux-last` = intégration, `main` = stable) et un commit type.
- **GitLab CI/CD** (`.gitlab-ci.yml`) : pipeline **4 étapes** `pretest` → `test` → `build` → `deploy` , runner tagué **raspberry** , déclenché sur la branche d'app → `docker compose build` puis `docker compose up -d` **directement sur le Pi**. C'est le **déploiement continu** réel.
- **Portainer** (`http://<pi>:9000`) : supervision graphique des conteneurs (logs, restart, état) — montrer les conteneurs UP.
- **Docker / docker-compose** : multi-services (app, portainer, ollama, ollama-pull), **volumes** (`./app/data:/data` = base persistante), **healthcheck**, image **multi-stage arm64**.

Phrase : « Le code est versionné sur Git/GitHub avec des commits atomiques ; une chaîne CI/CD GitLab construit et déploie automatiquement l'image Docker sur le Raspberry Pi ; Portainer me sert à superviser les conteneurs en production. »

4. Situer ma contribution dans le projet global (2 min)

Projet d'équipe de 8, 2 sous-équipes (JavaScript / Python). **Moi = E2.**

Membre	Rôle
E1 Bonnevey (Maxime)	Infrastructure, Docker, CI/CD, réseau du Pi
E2 Trompier (moi)	BDD, API, UI/UX, jeux solo + IA + multijoueur, mesure CS-160, documentation
E3 Arbadji (Ilyes)	Éditeur no-code de jeux
E4 Akyuz (Hasan)	Tests de l'API, fiches de recette

Phrase : « Je suis le **cœur applicatif** : la base de données, l'API, l'interface, les jeux et la mesure. Je m'appuie sur l'**infra de E1** (Docker/CI/Pi), j'expose l'**API que E4 teste**, et je fournis le **runtime de jeu** que l'**éditeur de E3** produit. »

5. Constituants de ma partie + interactions / tests d'intégration (3 min)

Mes constituants (ce que j'ai construit) :

- **Couche données** `lib/db/index.ts` — SQLite singleton, migrations idempotentes (WAL, busy_timeout, FK).
- **Auth** `lib/auth.ts` — PBKDF2-HMAC-SHA512, sessions cookie HttpOnly/SameSite.
- **API** `app/api/*` — `auth`, `classes`, `scores`, `games`, `multiplayer`, `p4`, `spectre`, `cs160`, `supervision`, `ai`, `admin`, `health`.
- **UI/jeux** `app/_components/*` — catalogue, Room3D, Color Speed, Puissance 4 (IA), Simon, Tetris, multijoueur, page Mesure.

Interactions (= les tests d'intégration entre nos parties) :

Frontière d'intégration	Qui ↔ qui	Comment je l'ai validée
Mon API ↔ tests de E4	E2 ↔ E4	E4 rejoue les fiches de recette sur mes routes (<code>/api/auth</code> , <code>/api/scores</code> ...) ; contrats JSON stables
Mon runtime de jeu ↔ éditeur de E3	E2 ↔ E3	Un jeu créé/généré dans l'éditeur E3 est exécuté par mon moteur ; format de jeu commun en base (<code>crg_games</code>)
Mon app ↔ infra de E1	E2 ↔ E1	Mon image se build dans sa chaîne Docker/CI ; <code>healthcheck</code> <code>/api/health</code> ; volume <code>/data</code>
Mon API ↔ matériel	E2 ↔ supervision/CS-160	Proxy <code>/api/supervision/batch</code> (sémaphore) ; pont <code>/api/cs160</code> (XYZ/Lv/x,y)

Phrase : « Mes interactions principales sont : l'**éditeur de E3** qui s'exécute sur **mon** moteur de jeu, **mes routes API** rejouées par les **tests de E4**, et **mon conteneur** intégré dans la **CI Docker de E1**. C'est là que se jouent les tests d'intégration. »

6. Montrer/expliquer mon code et son évolution (2 min)

4 extraits à ouvrir en live (les mêmes que dans le deck, slides 16/18/20/27) :

1. `app/api/auth/register/route.ts` — **variable transactionnelle** : `db.transaction(() => { ... })()` → user + adhésion classe, **tout-ou-rien** (BEGIN/COMMIT/ROLLBACK, ACID).
2. `app/api/supervision/batch/route.ts` — **variable atomique** : compteur `hwInFlight` + file de Promises (`acquireHwSlot` / `releaseHwSlot`), atomique car boucle d'événements **mono-thread**, borné à **2 slots**.

3. `lib/auth.ts` — `hashPassword` : `pbkdf2Sync(password, salt, 100_000, 64, 'sha512')` , format `sel:hash` . → *« jamais de mot de passe en clair. »*
4. `app/_components/GamePuissance4.tsx` — `scoreWindow` + `minimax` alpha-bêta. → *« l'intelligence de l'IA. »*

Évolution dans le temps (à raconter avec `git log`) :

- **Début** : scaffold Next.js + Docker, API jeux, auth en `localStorage`.
- **Refactor clé** : `feat(auth): move sessions to SQLite and remove localStorage` → passage à des **sessions serveur** en base (plus sûr).
- **Montée en puissance** : jeux (Color Speed, Puissance 4 + IA, Simon, Tetris color-match), **multijoueur** persisté, **mesure CS-160**, **IA locale Ollama** pour l'éditeur.
- **Durcissement** : `perf(3D)` (fin de l'écran noir, libération du contexte WebGL), `fix(jeux)` Simon (son/anti-race), garde-fous IA.
- **Industrialisation** : version affichée en console (v3.9 + date/heure), CI GitLab → déploiement Pi, documentation (15 diagrammes UML + guide).

Commande à montrer :

```
git log --oneline app/lib/auth.ts      # évolution d'un fichier précis
git log --oneline | wc -l              # ~280 commits = travail incrémental
```

Check-list finale (à cocher juste avant de passer)

- ☐ Pi allumé, `docker compose ps` = tout UP, `/api/health` OK
- ☐ CS-160 branché en USB et détecté
- ☐ Tablette + téléphone connectés au Wi-Fi **ColorRoom_WiFi**
- ☐ Compte enseignant **et** compte apprenant prêts
- ☐ Une partie multijoueur testée à 2 terminaux (le code fonctionne)
- ☐ **Vidéos de secours** prêtes (R1, R2, R3)
- ☐ `git log` ouvert dans un terminal, les 4 extraits de code ouverts dans l'éditeur
- ☐ Portainer ouvert dans un onglet

Le jury évalue : ta contribution personnelle, ta maîtrise technique, ton implication. À chaque test de recette, dis *« c'est moi qui ai développé cette partie »* et explique le **comment** (pas seulement le quoi).

</content>

</invoke>